

SISTEMA DE GESTÃO
ESCOLINHA
DE FUTEBOL

BRUNO C. BORGES

GUILHERME A. SILVA

JOÃO DRUMMOND

OBJETIVO

Atualmente, muitas escolinhas de futebol utilizam processos manuais e desconectados (planilhas, WhatsApp e anotações), gerando desorganização, falhas de comunicação e dificuldade no acompanhamento dos alunos.

O projeto propõe um sistema integrado para administradores, treinadores e responsáveis, centralizando gestão financeira, pedagógica e comunicacional, aumentando a eficiência e o profissionalismo da instituição.



ARQUITETURA



MVC (Modelo-Visão-Controlador)

- Separação Clara de Responsabilidades: Isola a interface gráfica (Visão - Pacote view com Swing), a lógica de negócio (Controlador - Pacote service) e os dados (Modelo - Pacotes domain e repository), facilitando o desenvolvimento paralelo e a manutenção.
- Preparação para Evolução: Mesmo implementado em Java Swing/Console para o T2, a estrutura MVC facilita uma futura migração para uma aplicação web (T3), reaproveitando as camadas de serviço e modelo.
- Manutenibilidade e Testabilidade: A separação permite testar a lógica de negócio (Serviços) independentemente da interface e facilita a modificação da interface sem impactar as regras de negócio.

SINGLETON



Garantir que exista apenas uma instância da classe ConfiguracaoApp, responsável por carregar e fornecer configurações globais da aplicação (como nome da escolinha), evitando múltiplas leituras e inconsistências.

Justificativa da Escolha: O Singleton é o padrão ideal para recursos que devem ser únicos e globalmente acessíveis na aplicação, assegurando um ponto centralizado de acesso às configurações.

Aplicação no Código: A classe ConfiguracaoApp foi implementada no pacote com.escolinha.config. O acesso em outras classes (ex: MainFrame) é feito via ConfiguracaoApp.getInstance().getPropriedade(...).

OBSERVER



Notificar a interface gráfica (MainFrame) de forma desacoplada quando um evento relevante ocorre na camada de serviço, especificamente quando um pagamento de fatura é registrado com sucesso no `FinanceiroService`.

O Observer permite que múltiplos objetos (Observers, como a `MainFrame`) sejam notificados automaticamente sobre mudanças de estado em outro objeto (Subject, como o `FinanceiroService`), sem que o Subject precise conhecer detalhes dos Observers. Isso promove baixo acoplamento e facilita a atualização da interface em resposta a eventos do back-end.

As interfaces `Observer` e `Subject` foram criadas no pacote `com.escolinha.observer`. `FinanceiroService` implementa `Subject` e chama `notifyObservers()` após registrar um pagamento. `MainFrame` implementa `Observer`, se registra no `FinanceiroService` e exibe um `JOptionPane` no método `update()`.

REPOSITORY

Isolar a camada de serviço (service) dos detalhes de como os dados são armazenados e recuperados (neste caso, em coleções Java em memória), permitindo maior flexibilidade para futuras mudanças na persistência (ex: banco de dados no T3).

O padrão Repository atua como uma abstração (semelhante a uma coleção) para o acesso aos dados de domínio (Aluno, Turma, etc.), centralizando as operações de CRUD e consultas. Isso desacopla a lógica de negócio da lógica de acesso a dados, melhora a testabilidade dos serviços (permitindo mocks dos repositórios) e a manutenibilidade geral do código. Classificado como Estrutural por organizar o acesso ao subsistema de dados.

Interfaces como `AlunoRepository`, `TurmaRepository`, etc., foram definidas no pacote `com.escolinha.repository`. As implementações concretas (ex: `AlunoRepositoryMemoria`) que utilizam `Map` ou `List` foram criadas no mesmo pacote. As classes de serviço no pacote `com.escolinha.service` dependem exclusivamente das interfaces dos repositórios, recebendo as implementações via injeção de dependência no construtor.

